SLiM Workshop Series #15: Chromosomes, Haploidy, Sex

Copyright © 2020–2025 Benjamin C. Haller. All rights reserved.

1. Explicitly defining a chromosome

Close all windows in SLiMgui. Then make a new WF model with command-N, or go to SLiMgui's File menu and choose **New**. Modify the code to look like this:

```
initialize() {
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);

   initializeChromosome(1, 1e5);
   initializeGenomicElement(g1);
   initializeMutationRate(1e-7);
   initializeRecombinationRate(1e-8);
}
1 early() {
   sim.addSubpop("p1", 500);
}
2000 late() { sim.outputFixedMutations(); }
```

There are a few things to notice here. In a multi-chromosome model, all chromosomes share the same set of defined mutation types and genomic element types, so it makes sense to define those first, before getting into defining the chromosomes. After that, this model explicitly defines a single chromosome with `initializeChromosome()`. Click between the parentheses of that call to see the full function signature, at the bottom of SLiMgui's window:

```
(object<Chromosome>$)initializeChromosome(integer$ id,
   [Ni$ length = NULL], [string$ type = "A"], [Ns$ symbol = NULL], ...)
```

We'll look at the parameters shown above in the workshop; there are a couple more that we won't use, but of course they're described in the manual. Above we've defined a chromosome with an `id` of `1` and a `length` of `1e5`. Since we now have an explicitly defined chromosome length, we don't need to specify the range for the genomic element; we can just call `initializeGenomicElement(g1)`, and by default it creates a genomic element that spans the full length of the chromosome. Then we set mutation and recombination rates as usual; the rest of the model is unsurprising. Recycle and run just to make sure the model is working.

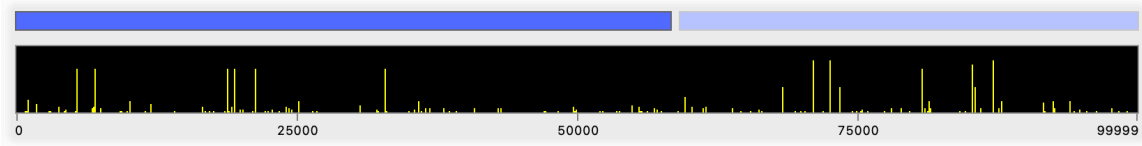
2. Defining a second autosome

Let's define a second diploid autosome, of a slightly different length. Copy and paste the four lines that define the chromosome structure, adding these lines to the end of the `initialize()` callback:

```
   initializeChromosome(2, 7e4);
   initializeGenomicElement(g1);
   initializeMutationRate(1e-7);
   initializeRecombinationRate(1e-8);
```

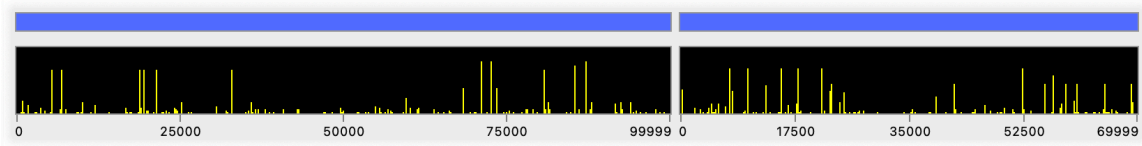
That's all that's needed to make a model with two diploid autosomes. Notice that the initialization of the first chromosome follows the first `initializeChromosome()` call, and the initialization of the second chromosome follows the second call; each call to `initializeChromosome()` begins the configuration of the next chromosome. The structure of the initialization is thus naturally grouped into blocks, setting up one chromosome at a time. It's nice to put a blank line between each of those blocks to make the structure of the model more visible.

Recycle and run this model, and in SLiMgui you should see something like this:



The chromosome overview – the top strip – shows both chromosomes, but the highlighting shows that only the first chromosome is selected for display. The chromosome detail view – the bigger bottom half of the display – therefore shows details for the selected chromosome (the first one).

Click on the first chromosome in the overview. Since it was already selected, this switches to selecting *all* of the chromosomes, producing a display like this:



The overview now shows both chromosomes as selected, and the detail view shows details for both.

EXERCISE: Try clicking several times to select and deselect chromosomes, just to get used to how the view works. There are a couple of other things you can try as well. (1) Hover the cursor over the chromosome overview (i.e., place the cursor over the overview and then don't move it at all for a second or two). Notice how the overview switches to displaying the id of each chromosome (the symbol, actually, but by default the symbol is the same as the id). This can be quite useful for finding the right chromosome if you have a lot of them displayed! (2) Click and drag in the chromosome overview. Notice how you have now selected a subrange of a chromosome for display in the detail view, zooming in on the selected section. A simple click in the overview will zoom back out again. Play around with these different display modes a little.

3. Separate sexes

We'd like to model separate sexes, on the way to modeling sex chromosomes. Doing that is quite trivial; just add this line at the top of the `initialize()` callback:

```
initializeSex();
```

This is all that is needed; the model now has males and females, and each reproductive cross is between a female first parent and a male second parent. If you recycle and run, the model looks much the same, but you can tell that it's a sexual model by looking at the population table:

ID	N	↺	♁♀	♁♂	◕
p1	500	—	0.00	0.00	0.50

The last column, with the yin-yang symbol as its header, shows the sex ratio; by default it is `0.5`. (The previous three columns show, from left to right, the selfing rate, the female cloning rate, and the male cloning rate.) So now we have a sexual model!

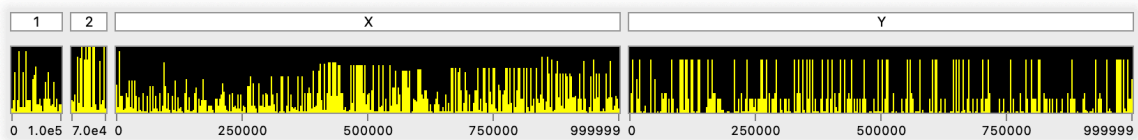
4. Sex chromosomes

Next let's add sex chromosomes to this model. This just involves defining a couple of additional chromosomes. So far we haven't explicitly specified the type of the chromosomes; we've just used the default type of `"A"`, producing a diploid autosome. Now, however, we'll provide explicit types. Add these lines to the end of the `initialize()` callback:

```
   initializeChromosome(3, 1e6, "X", "X");
   initializeGenomicElement(g1);
   initializeMutationRate(1e-7);
   initializeRecombinationRate(1e-8);

   initializeChromosome(4, 1e6, "Y", "Y");
   initializeGenomicElement(g1);
   initializeMutationRate(1e-7);
```

This makes X and Y chromosomes, simply by passing `"X"` and `"Y"` as the chromosome types in the third parameter. (We also pass `"X"` and `"Y"` for the chromosome symbols in the fourth parameter, to make the display in SLiMgui nicer as we'll see momentarily, but that is purely optional). These chromosomes are defined to be relatively long, compared to the autosomes, so that we can see them clearly when we recycle and run. Here's a screenshot, hovering the mouse cursor over the chromosome overview so it displays the chromosome symbols:



The difference in the evolutionary dynamics of the X and Y is apparent here, although it isn't always so visible. Mutations on the X have a wide variety of frequencies, because recombination is constantly scrambling things up. The Y, on the other hand, shows two predominant haplotypes across its entire length, one at perhaps 80% frequency, the other at perhaps 20%. This is because the Y is inherited clonally, whereas the X recombines in females; it's neat that the evolutionary consequences of that are so easily visualized. (Note that in the code above, we don't set a recombination rate for the Y; it can be omitted for haploid chromosomes, and then `0.0` is assumed. In fact, even if we specified a recombination rate for the Y, it wouldn't be used; SLiM knows that the Y is inherited clonally through the male line.)

EXERCISE: Remember the "Display Haplotypes" mode for the chromosome view, selectable through its action button on the right? Try using that now to examine the difference in haplotype structure between the X and the Y more closely. Run the model a couple of times to get a sense for how the evolutionary dynamics of the X and Y are different. (The speed slider might be useful too.)

5. Using a joint `for` loop to simplify multi-chromosome initialization

This model now has a lot of initialization code to set up its four chromosomes. Much of that is repetitive; it would be nice to simplify it. We can do that by enclosing the initialization code in a `for` loop. Try changing the `initialize()` callback to match this code:

```
initialize() {
   initializeSex();
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);

   for (id in 1:4, length in c(1e5, 7e4, 1e6, 1e6),
      type in c("A", "A", "X", "Y"))
   {
      initializeChromosome(id, length, type);
      initializeGenomicElement(g1);
      initializeMutationRate(1e-7);
      initializeRecombinationRate(1e-8);
   }
}
```

This `for` loop syntax is called a "joint `for` loop". It loops over the given values for `id`, `length`, and `type` in synchrony; the loop runs four times, using the first value for each of the index variables first, the second value for each second, and so forth. In this manner, it creates all four chromosomes. Make sure you understand how this works; it's a very useful language feature in Eidos!

EXERCISE: As it stands, this version of the model is actually slightly different, because we're not defining the chromosome symbols any more. If you hover over the overview, you'll see that the labels for the X and are are simply "3" and "4", sadly. Modify the joint `for` loop so symbols are defined for the chromosomes as well; you might use symbols `"A1"`, `"A2"`, `"X"`, and `"Y"` for them.

EXERCISE: So far, all the chromosomes have the same mutation rate of `1e-7`. Extend the joint `for` loop again, to give each chromosome a different mutation rate, and observe the outcome.

In SLiM, every individual in a species has the same number of haplosomes but some haplosomes might be "null" haplosomes that act as placeholders, occupying an unused space. That commonly occurs when modeling sex chromosomes, since their ploidy varies by sex. In this model, every individual possesses two X haplosomes and one Y haplosome; in SLiM terminology, the X is "intrinsically diploid" (two haplosomes per individual, always), and the Y is "intrinsically haploid" (one haplosome per individual, always). The unused haplosomes in an individual of a given sex are null haplosomes; we could say that males in this model are X– Y and females are XX –, where – represents a null haplosome. We can examine how this works in the Eidos console, after running the model:

```
> p1.sampleIndividuals(1, sex="M").haplosomes
Haplosome<A:25> Haplosome<A:17> Haplosome<A:17> Haplosome<A:14>
Haplosome<X:75> Haplosome<X:null> Haplosome<Y:53>
> p1.sampleIndividuals(1, sex="F").haplosomes
Haplosome<A:17> Haplosome<A:21> Haplosome<A:10> Haplosome<A:12>
Haplosome<X:65> Haplosome<X:74> Haplosome<Y:null>
```

We sample a male and a female, and print their haplosomes. The way SLiM prints objects like haplosomes is mostly designed for debugging; in this case, the chromosome type (`A`, `X`, or `Y`) is printed before the colon, and the number of mutations the haplosome contains is printed after (or "null" if it is a null haplosome). Both individuals contain seven haplosomes: two for the first diploid autosome, two for the second, two for the X, and one for the Y. In the male, the second X haplosome is a null haplosome; in the female; the Y haplosome is a null haplosome. In general you are not allowed to manipulate null haplosomes; they are inert placeholders. You can test whether a given haplosome is a null haplosome with the `isNullHaplosome` property.

6. A model of clonal haploids

We could take the above model further, adding genetic structure within each chromosome, adding more chromosomes to the model, and so forth; you can see how you might build a whole-genome model this way. But let's switch gears to quickly touch upon a different topic: modeling haploids.

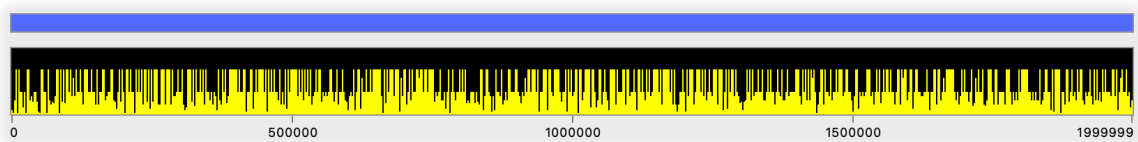
Open a new SLiMgui window and edit it down to the following model:

```
initialize() {
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);

   initializeChromosome(1, 2e6, "H");
   initializeGenomicElement(g1);
   initializeMutationRate(1e-7);
   initializeRecombinationRate(1e-8);
}
```

```
1 early() {
   sim.addSubpop("p1", 2000);
   p1.setCloningRate(1.0);
}
10000 late() { sim.outputFixedMutations(); }
```

We pass "H" for the chromosome type now, requesting a haploid autosome. If you recycle and run, you should observe clonal dynamics in the model, similar to the Y chromosome's dynamics earlier:



The `setCloningRate(1.0)` call makes reproduction entirely clonal even though a recombination rate is set; so this is a simple model of clonal haploids such as bacteria.

EXERCISE: You might wonder what happens if the `setCloningRate()` call is commented out. Try it! Can you see the difference in the evolutionary dynamics? Chromosome type "H" actually supports haploid recombination with biparental crosses. This can be a good model of the biology of some systems, such as bryophytes; this is discussed in detail in section 8.3.6 of the manual.

7. Switching to nonWF

Thus far we've looked at WF models for simplicity, but multiple chromosomes can be configured in nonWF models in precisely the same way. Let's convert the previous model into a nonWF model:

```
initialize() {
   defineConstant("K", 2000);
   initializeSLiMModelType("nonWF");
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);
   m1.convertToSubstitution = T;

   initializeChromosome(1, 2e6, "H");
   initializeGenomicElement(g1);
   initializeMutationRate(1e-7);
   initializeRecombinationRate(1e-8);
}
reproduction()
{
   p1.addCloned(individual);
}
1 early() {
   sim.addSubpop("p1", K);
}
early() {
   p1.fitnessScaling = K / p1.individualCount;
}
10000 late() { sim.outputFixedMutations(); }
```

The conversion simply involves adding a `reproduction()` callback and adding density-dependent fitness to provide population regulation. We also set `convertToSubstitution` to `T` for `m1`, since those mutations wouldn't fix and be converted to substitutions automatically any more. The way that haploidy is configured, with the chromosome of type "H", is unchanged.

However, you might have noticed that mutations seem to take longer to fix in this nonWF model. This is because it has overlapping generations and age structure, unlike the WF model. The mean

lifetime is longer, and so the time to fixation in ticks is longer. In some cases this might be biologically realistic and desirable; in other cases, however, non-overlapping generations might be better. How can we switch this nonWF model to use non-overlapping generations?

8. Adding non-overlapping generations

Simply killing off the parental generation each tick is not sufficient to make an evolutionary model with non-overlapping generations. For example, let's replace the `early()` event that generates density-dependent selection with a new version:

```
early() {
   inds = sim.subpopulations.individuals;
   inds[inds.age > 0].fitnessScaling = 0.0;
}
```

But this wouldn't evolve! Each individual would have one offspring (from the `reproduction()` callback above), and that one offspring would be guaranteed to survive and produce one offspring of its own, so in effect, we would have `2000` completely independent clonal lineages, each containing one individual in every generation. Apart from new mutations added to the population in each tick (and then never changing if frequency thereafter), nothing would change, because there would be no competition between lineages, and no stochasticity to produce drift. (You can run it to see!)

So we need a source of stochasticity. In a neutral Wright–Fisher model this comes from random mating; some individuals end up as parents multiple times, while others have no offspring, just by chance. We can do that here too, producing 2000 clonal offspring from 2000 randomly chosen parents in a Wright–Fisher type of approach. That's quite straightforward to implement; in addition to the new `early()` event shown above, we just need a new `reproduction()` callback that chooses parents for cloning randomly from the population:

```
reproduction()
{
   parents = p1.sampleIndividuals(K, replace=T);

   for (parent in parents)
      p1.addCloned(parent);

   // we've done the reproduction for this tick, so deactivate ourselves
   self.active = 0;
}
```

This is an example of "big bang" reproduction, as seen in the previous worksheet. First we choose all of the parents to be cloned, sampling them at random from the population. (We use the `sampleIndividuals()` method here; it would be equivalent to use `sample()` to draw a sample from `p1.individuals`, but would be a little slower.) Then we loop through the chosen parents and clone each one. We end by deactivating ourselves for the rest of the tick, since all reproduction is done.

This is just one possible approach. Another approach would be to keep the reproduction deterministic, and add stochasticity in the form of random mortality instead. For example, we could make every individual produce two offspring per tick, pushing the population above its carrying capacity. If we then apply density-dependent selection probabilistically to the juveniles (after killing off the parental generation), that would also add the stochasticity we need. The fundamental point here is that nonWF models are in charge of their own reproduction and population regulation, which is very powerful and flexible, but which raises many subtle questions. Different choices might lead to different evolutionary dynamics, so it's very important to think carefully about these things.

8. BEEP! With extra time, look at section 8.3.5, on modeling the full human genome in SLiM.